SE4050 – Deep Learning

Week 3 – Backpropagation, Loss function Recap + Machine Learning Basics

Dr. Mahima Weerasinghe, *MIET*

Content

- What mathematics components your require for DL?
- Ex 1 – Backpropagation calculation
- Loss (Cost) function
- Ex 2 – Loss functions
- Optimization algorithms
- Ex 3 – Optimization algorithms

Tips for getting started with DL

https://machinelearningmastery.com/5-tips-for-getting-started-with-deep-learning/#:~:text=The%20base%20of%20Deep%20Learning,Convolutional%20Neural%20Networks%20(CNNs)

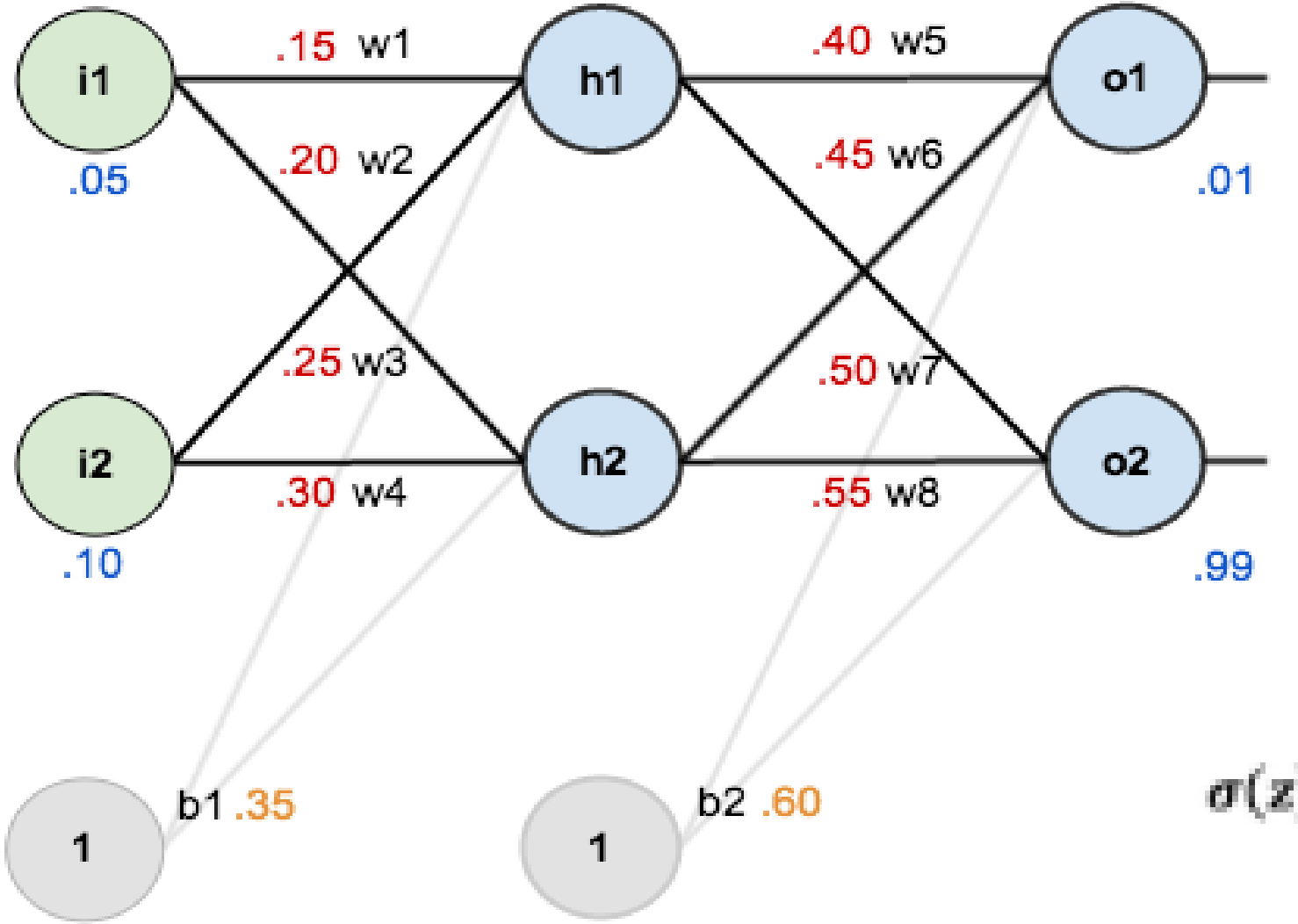
Mathematics requirement for Deep Learning

1. **Linear Algebra**
    - **Vectors and Matrices**: Understanding basic operations such as addition, multiplication, and the properties of matrices.
    - **Eigenvalues and Eigenvectors**: Useful for understanding data transformations and principal component analysis (PCA).
    - **Matrix Multiplication**: Essential for understanding how neural networks process data in layers.
2. **Calculus**
    - **Derivatives and Differentiation**: Fundamental for understanding how changes in weights affect the loss function.
    - **Chain Rule**: Critical for backpropagation in neural networks.
    - **Optimization**: Understanding gradient descent and its variants (SGD, Adam).

3. **Probability and Statistics**
    - **Probability Distributions**: Knowledge of distributions like Gaussian, Bernoulli, etc., to understand data and model outputs.
    - **Bayesian Theorem**: Important for probabilistic models and Bayesian neural networks.
    - **Expectation and Variance**: Key statistical measures that underpin many machine learning algorithms.
4. **Optimization Theory**
    - **Loss Functions**: Understanding different types of loss functions (e.g., mean squared error, cross-entropy) and their roles.
    - **Gradient Descent**: The mechanism behind how models learn.
    - **Regularization**: Techniques like L1, L2 regularization, dropout to prevent overfitting.

5. **Information Theory**
    - **Entropy and Cross-Entropy**: Understanding these concepts is crucial for evaluating classification models.
    - **KL-Divergence**: Used in advanced topics like variational autoencoders.
6. **Neural Network Fundamentals**
    - **Structure and Activation Functions**: Basics of neural network architecture, including activation functions like ReLU, sigmoid, and tanh.
    - **Forward and Backward Pass**: Understanding how data flows through the network and how gradients are propagated back.

Ex 1 – Calculate the new weights after single backprop



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

$$out_{h2} = 0.596884378$$

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.1059059967}} = 0.75136507$$

$$out_{o2} = 0.772928465$$

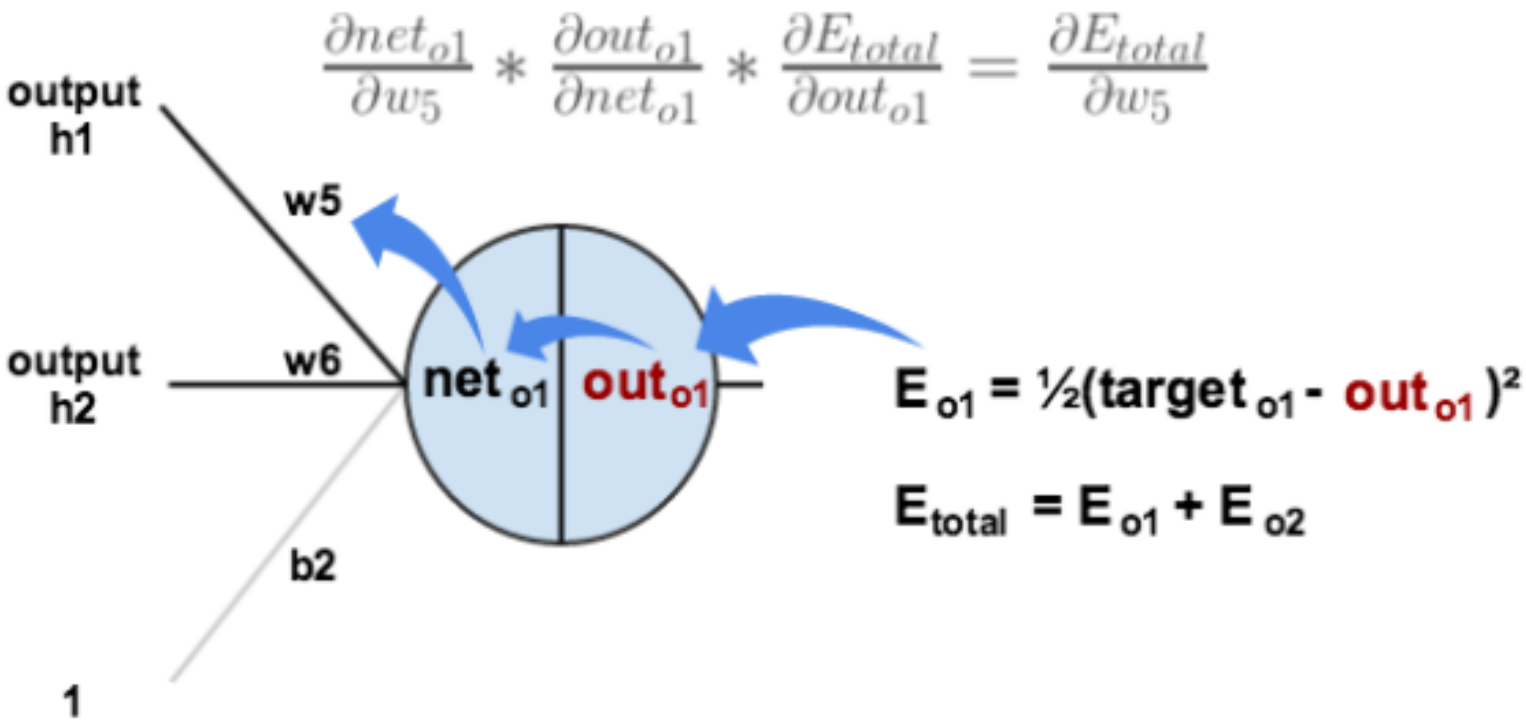
$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

Backward pass

Chain Rule

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$



$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

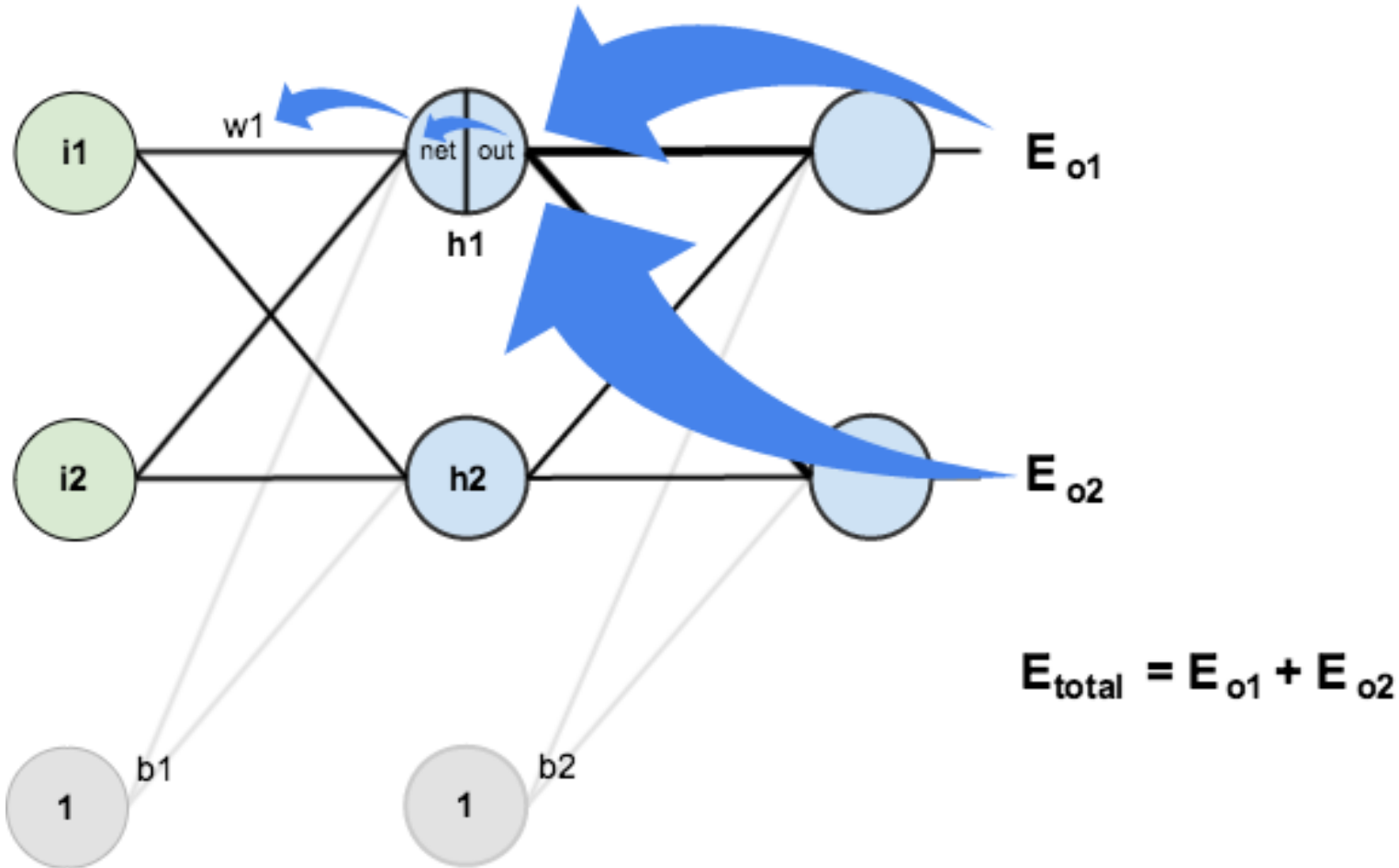
$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



$$E_{total} = E_{o1} + E_{o2}$$

Exercise 2

Understanding loss function - https://drive.google.com/file/d/1VgBd81LK8D1YqpzRXAsvoHQvybMpSseR/view?usp=sharing

- What is a loss function in a neural network ?
- What are the different loss functions used for different tasks (e.g. – Classification , Regression etc;)?
- Why do we might have to customize loss functions? Explain in depth

Loss function vs Optimization functions

Loss Function

Definition:

A loss function (or cost function) is a measure of how well the predictions of a machine learning model match the actual data. It quantifies the error between the predicted output and the true output.

Purpose:

- The loss function is used to evaluate the performance of a model.
- It provides a single scalar value that indicates how far off the model's predictions are from the actual targets.
- Common loss functions include Mean Squared Error (MSE) for regression tasks and Cross-Entropy Loss for classification tasks.

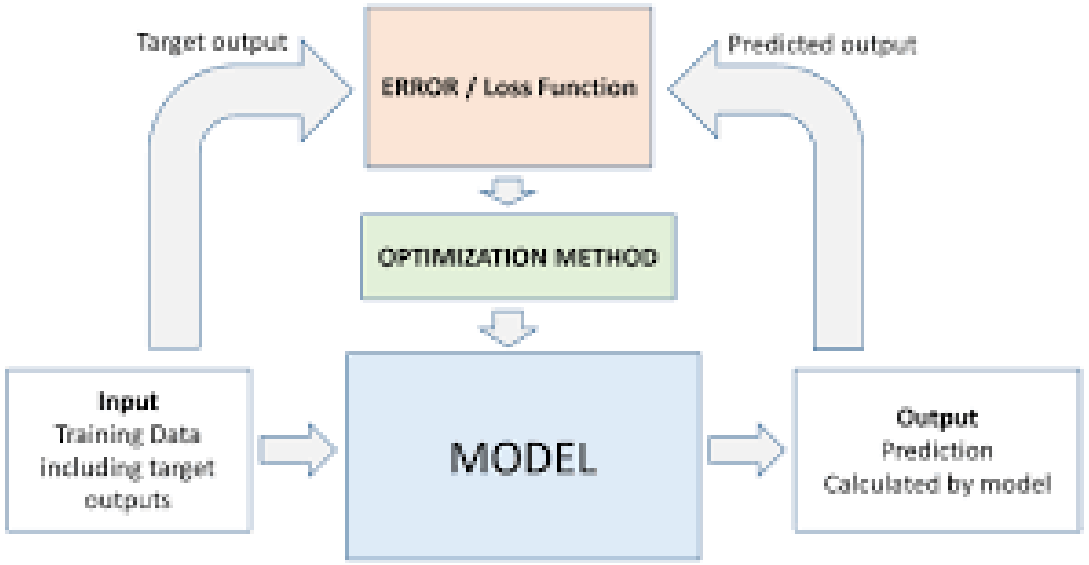
Gradient Descent

Definition:

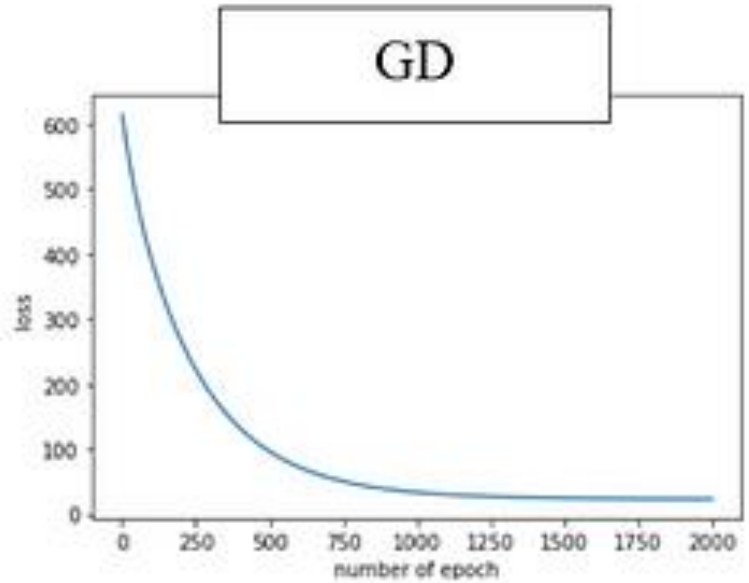
Gradient descent is an optimization algorithm used to minimize the loss function by iteratively adjusting the model parameters (weights and biases) in the direction that reduces the loss.

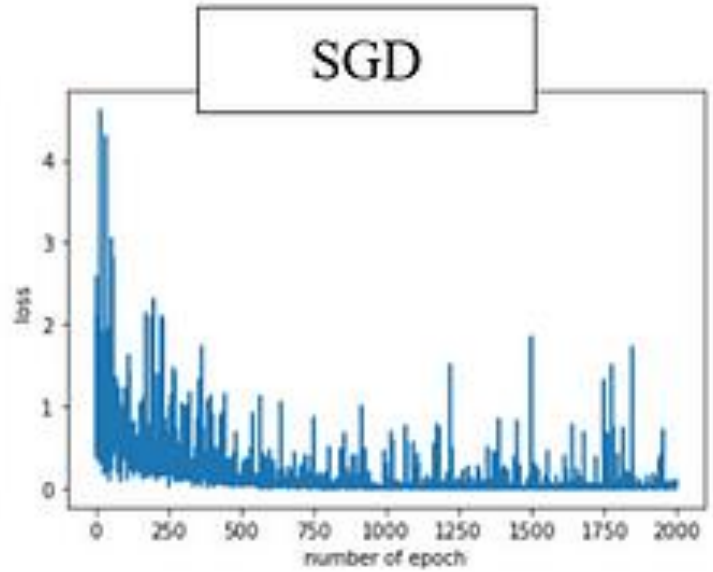
Purpose:

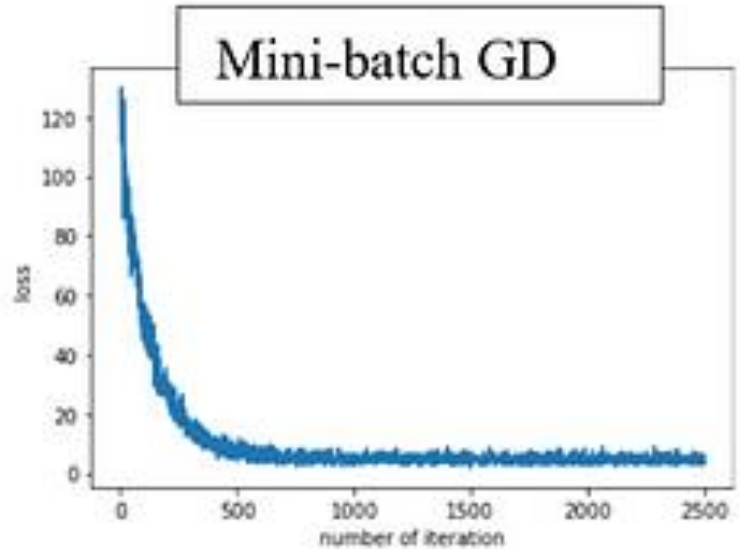
- Gradient descent helps find the optimal parameters that minimize the loss function.
- It adjusts the parameters by computing the gradients (partial derivatives) of the loss function with respect to each parameter and updating the parameters in the opposite direction of the gradients.



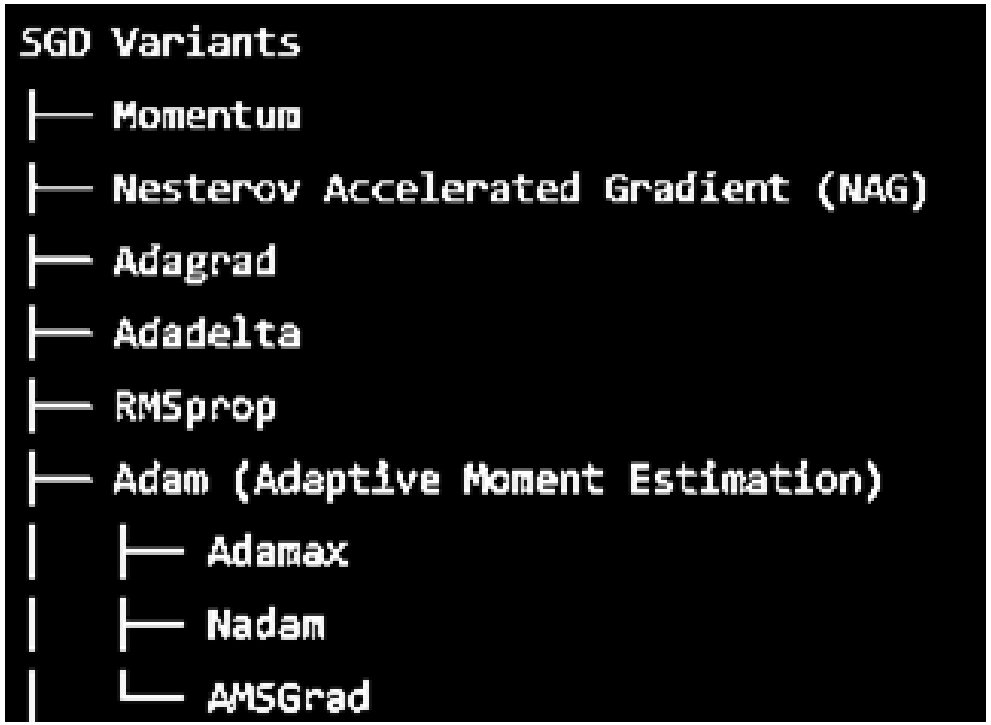
Aspect	Gradient Descent (GD)	Stochastic Gradient Descent (SGD)	Mini-batch Gradient Descent (MGD)
Data Usage	Entire dataset at each iteration	Single example at each iteration	Small batch at each iteration
Update Frequency	Infrequent updates (once per epoch)	Frequent updates (once per example)	Moderately frequent updates (once per mini-batch)
Convergence Path	Smooth, steady, but can get stuck in local minima	Noisy, fluctuating, can escape local minima	Balance between smoothness and noise
Convergence Speed	Slower due to using the entire dataset	Faster per iteration due to using a single example	Faster than GD but slower than SGD per iteration
Computation	Computationally expensive and slow for large datasets	More efficient and faster per iteration for large datasets	More efficient than GD, moderately fast for large datasets
Memory Usage	Requires storing entire dataset in memory	Requires less memory, processes one example at a time	Requires less memory than GD, processes small batches
Stability	More stable and less variance in updates	Less stable with high variance in updates	Moderately stable, less variance than SGD
Suitability	Small to medium-sized datasets	Large datasets	Large datasets
Variants	None	None	Combines advantages of both GD and SGD
Example Batch Size	Entire dataset (e.g., 100,000 examples)	1 example	Small batch (e.g., 32, 64, 128 examples)
Iteration Time	Long	Short	Moderate


GD
loss
number of epoch
0
100
200
300
400
500
600
0
250
500
750
1000
1250
1500
1750
2000


SGD
loss
number of epoch
0
1
2
3
4
0
250
500
750
1000
1250
1500
1750
2000


Mini-batch GD
loss
number of iteration
0
20
40
60
80
100
120
0
500
1000
1500
2000
2500

Variations of SGD



Nestrov: https://proceedings.mlr.press/v28/sut...
AdaGraD: https://www.jmlr.org/papers/volume12/...
AdaDelta: https://arxiv.org/abs/1212.5701
Adam & AdaMax: https://arxiv.org/abs/1412.6980
AMSGrad: https://arxiv.org/abs/1904.03590
AdaBound: https://arxiv.org/abs/1902.09843
AdamW: https://arxiv.org/pdf/1711.05101v2.pdf
Yogi: https://proceedings.neurips.cc/paper_...
Nadam: https://openreview.net/pdf/OM0jvwB8jI...

Lookahead: https://arxiv.org/abs/1907.08610

Exercise 3

- For which type of application is GD, SGD and MbSGD is most suitable to use ? Explain why ?

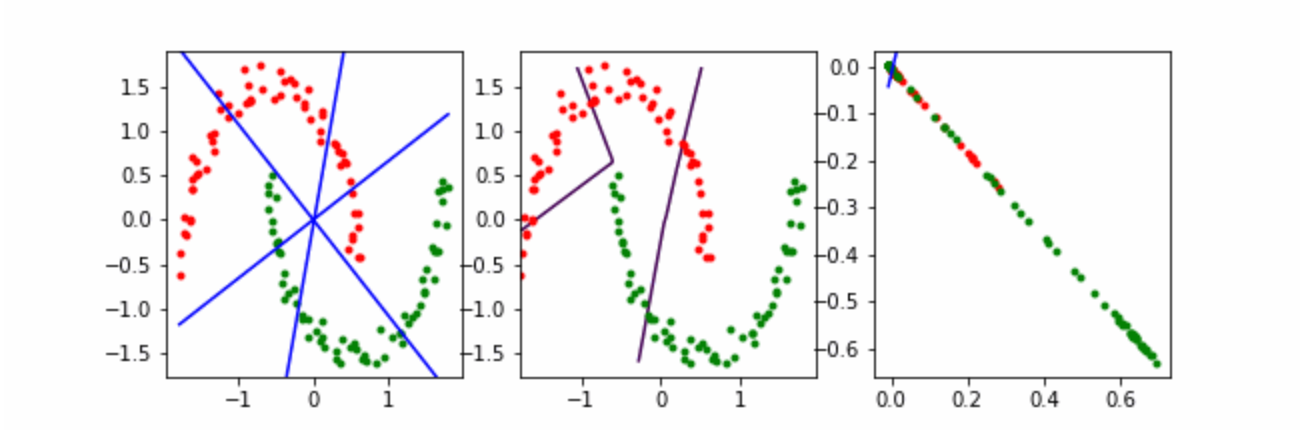
More about optimizers – https://builtin.com/data-science/gradient-descent#:~:text=Gradient%20Descent%20is%20an%20optimization,function%20as%20far%20as%20possible.

https://www.youtube.com/watch?v=7m8f0hP8Fzo

Regularization

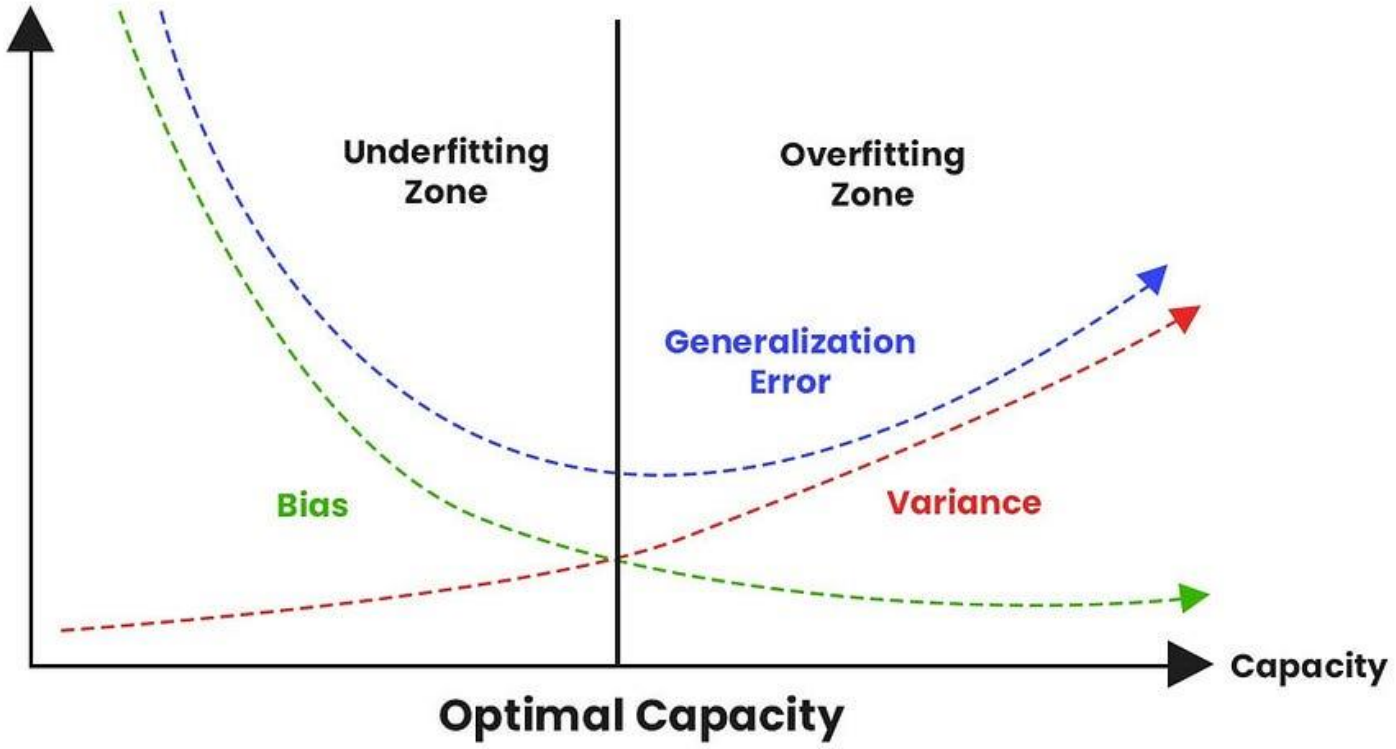
Goal of Deep Learning (& ML) - Making an algorithm that performs equally well on training data and any new samples or test dataset

Regularization - Regularization may be defined as any modification or change in the learning algorithm that helps reduce its error over a test dataset, commonly known as generalization error but not on the supplied or training dataset



https://medium.com/analytics-vidhya/regularization-understanding-l1-and-l2-regularization-for-deep-learning-a7b9e4a409bf

Regularization in Deep Learning



Bias

Bias refers to the error introduced by approximating a real-world problem, which may be complex, by a simplified model. It represents the assumptions made by the model to simplify the learning process.

Variance:

Variance refers to the model's sensitivity to fluctuations in the training data. A model with high variance pays too much attention to the training data, capturing noise and irrelevant details.

L1 Regularization

L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator), works by adding a penalty proportional to the absolute values of the weights to the loss function. Mathematically, if $\mathbf{w}$ represents the weights of the model, the L1 penalty term is:

$$\lambda \sum_i |w_i|$$

Here, λ is the regularization parameter that controls the strength of the penalty.

Effect:

- **Sparsity**: L1 regularization promotes sparsity in the weight vector. This means it tends to push some weights to exactly zero. This can be beneficial for feature selection, as features corresponding to zero weights are effectively excluded from the model.
- **Model Interpretation**: By zeroing out some weights, L1 regularization can make the model easier to interpret, as it uses fewer features.

L2 Regularization

L2 regularization, also known as Ridge regression, adds a penalty proportional to the square of the weights to the loss function. The L2 penalty term is:

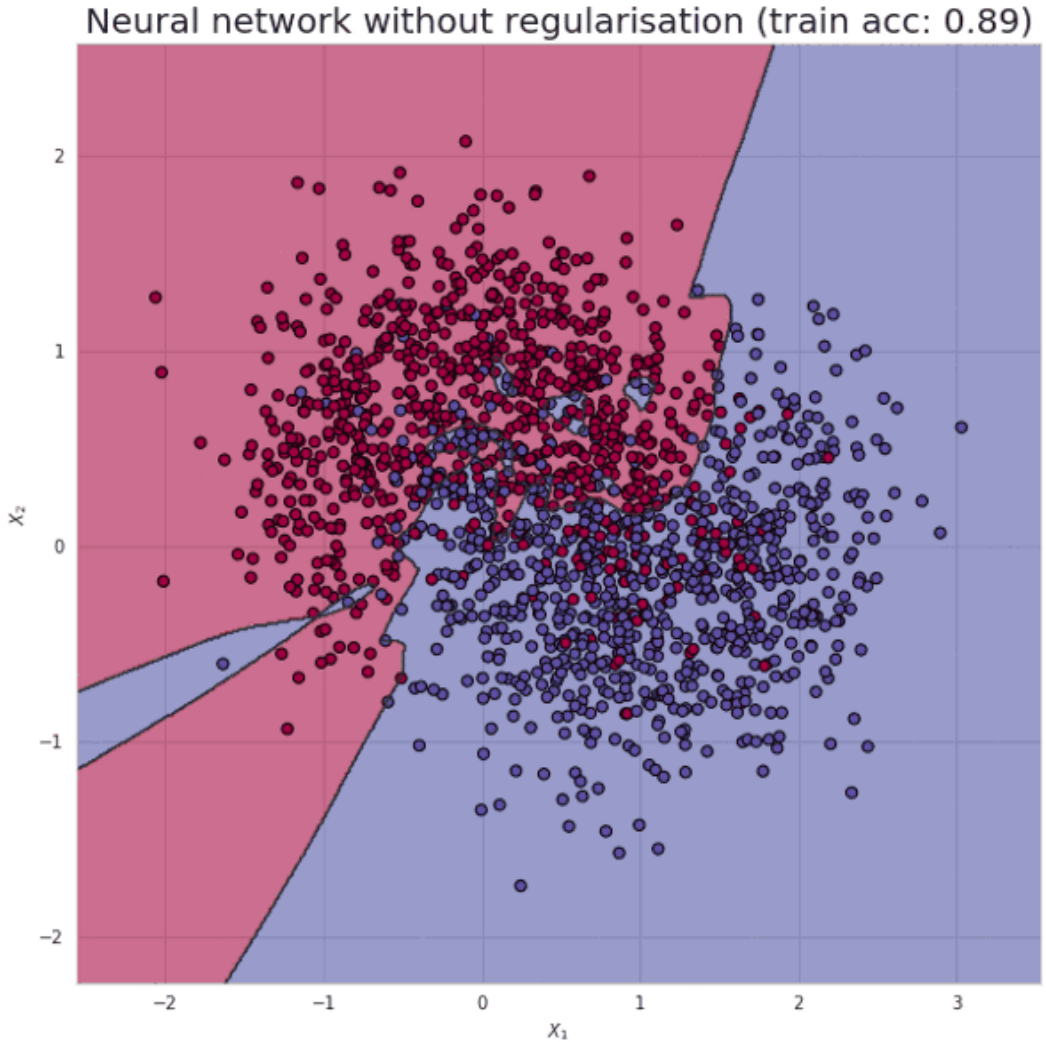
$$\lambda \sum_i w_i^2$$

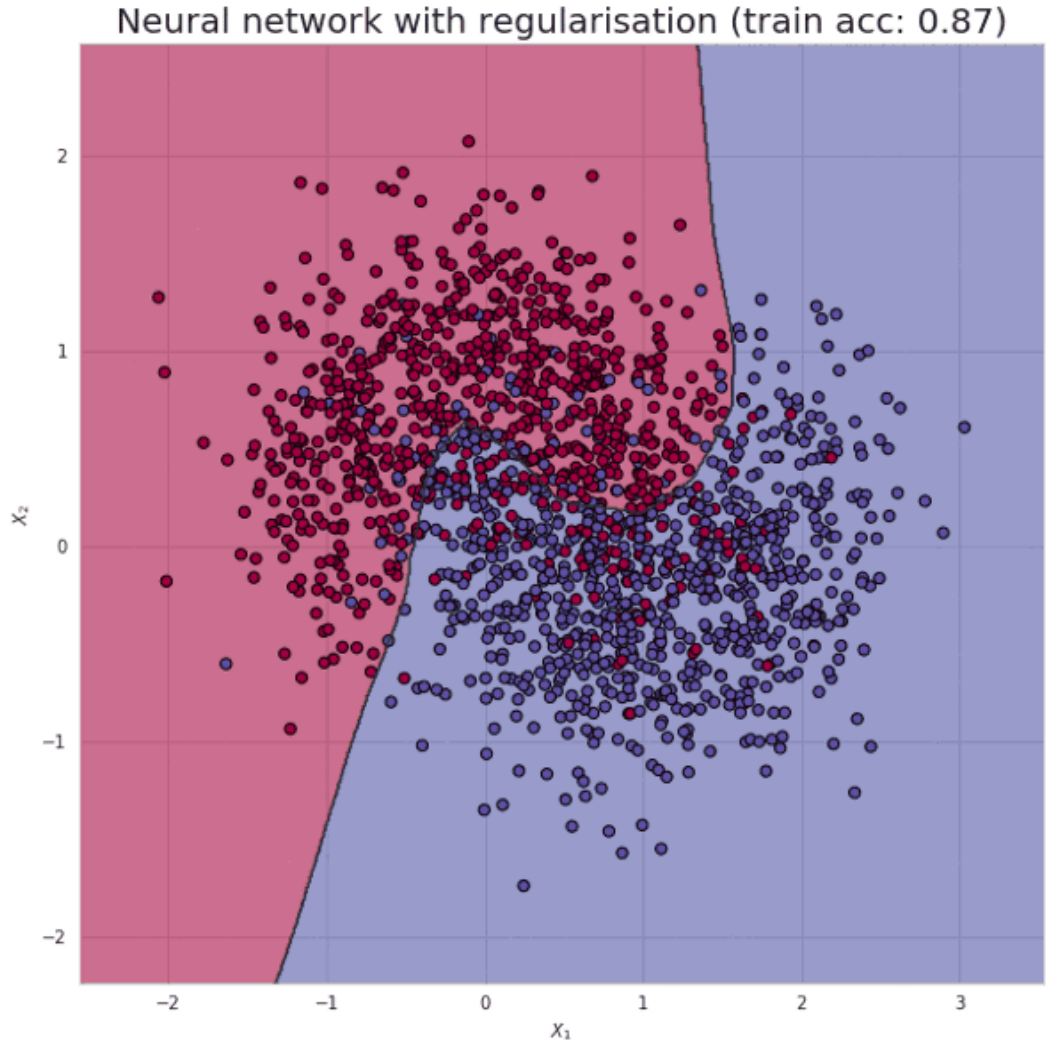
Again, λ controls the amount of regularization applied.

Effect:

- **Weight Shrinkage**: L2 regularization shrinks the weights towards zero but doesn't typically zero them out completely. It encourages weights to be small and evenly distributed, reducing the impact of any single feature.
- **Numerical Stability**: By penalizing large weights, L2 regularization can improve the numerical stability of the model, helping to avoid overfitting and making the model more robust to variations in the training data.

L1 Regularisation	L2 Regularisation
Sum of absolute value of weights	Sum of square of weights
Sparse solution is the outcome	Non-sparse (more segregated) solution
Multiple solutions are possible	Only one solution
Built-in feature selection in the penalty term	No specific feature selection mechanism
Robust to outliers	Not robots to outliers due to square term
Used in datasets with sparse features	Used in dataset with complex features


Neural network without regularisation (train acc: 0.89)
2
1
0
−1
−2
X_2
−2
−1
0
1
2
3
X_1


Neural network with regularisation (train acc: 0.87)
2
1
0
−1
−2
X_2
−2
−1
0
1
2
3
X_1

Basics of Machine Learning

- Refer to the additional Lecture provided

Resources

- Deep Learning from Scratch

https://colab.research.google.com/github/alvinntnu/NTNU_ENC2045_LECTURES/blob/main/nlp/dl-neural-network-from-scratch.ipynb#scrollTo=jb6VI1BfWshk

Regularisation

https://github.com/codebasics/py/blob/master/ML/16_regularization/L1%20and%20L2%20Regularization.ipynb